



Faculty of Engineering, Built Environment and Information Technology

Fakulteit Ingenieurswese, Bou-omgewing en
Inligtingtegnologie / Lefapha la Boetšenere,
Tikologo ya Kago le Theknolotši ya Tshedimošo

Make today matter

www.up.ac.za

DEPARTMENT OF COMPUTER SCIENCE

COS 122 OPERATING SYSTEMS

Practical 0

Due: End of Semester

PLAGIARISM POLICY

UNIVERSITY OF PRETORIA

The Department of Computer Science considers plagiarism as a serious offence. Disciplinary action will be taken against students who commit plagiarism. Plagiarism includes copying someone else's work without consent, copying a friend's work (even with consent) and copying material (such as text or program code) from the Internet. Copying will not be tolerated in this course. For a formal definition of plagiarism, the student is referred to <https://www.library.up.ac.za/plagiarism> (from the main page of the University of Pretoria site, follow the *Library* quick link, and then *Services* and then click the *Plagiarism* link). If you have any form of question regarding this, please ask one of the lecturers, to avoid any misunderstanding. Also note that the OOP principle of code re-use does not mean that you should copy and adapt code to suit your solution. The use of generative AI such as Gemini/Claude/ChatGPT is prohibited within the context of this practical and will result in plagiarism charges.

Objectives

This practical explores the submission of code to Fitchfork using processes in the C++ programming language. The concept of processes is covered in chapter 2 of the prescribed textbook. This practical has 1 task for a total of 3 marks.

Upload Instructions

You need to write C++ code for the task. You are then required to submit your code on Fitchfork where it will be marked automatically.

- Upload your task in a single zip or tar.gz file to the respective Practical 0 assignment slot on Fitchfork before the end of the semester. **No late submissions will be accepted!**
- Make use of a Makefile for your program.
- Your Makefile needs to use the `g++` compiler.
- Your Makefile **must** have at least these commands:
 - `# make`
 - `# make run`
- Only upload your source code (.h and .cpp files) and makefiles. Do not upload large, binary or OS files. Keep your uploaded archive small.
- **Failure to upload your work will result in 0 marks being awarded for your practical!**

General Instructions

For this practical, you will need to use some kind of C++ development environment. You can set up a Linux or WSL environment on your own machine (if you don't already have one for COS132/COS110). See the additional documentation for various possible options.

- You are not allowed to use external libraries. Only standard C and C++ libraries may be imported.
- Every file you submit should contain your student number at the top of the file (source code and makefiles).
- If your program does not have a **makefile** you will get 0 marks.
- If your program doesn't compile due to syntax errors you will get 0 marks.
- If your program receives a runtime error you will lose marks.

Useful Links

Linux man pages

<https://www.man7.org/linux/man-pages/man2/fork.2.html>
<https://www.man7.org/linux/man-pages/man2/wait.2.html>
<https://www.man7.org/linux/man-pages/man3/sleep.3.html>

Task 1 - Concurrent Process Execution [3 marks]

For this task you will be required to start multiple processes that will each perform separate tasks. The key here is to show that the processes are indeed running concurrently.

Concurrent Process Execution Implementation

The file `process_main.cpp` declares all the necessary functions and global variables needed to implement this task.

Your task

You will need to implement the following functions in `process_main.cpp`:

- `void parentProcess();`
Purpose: This where the parent process will do its work. You need to print out "Parent process executing" **five** times. Add a delay between each printout using the `sleep()` function. A suitable delay will cause interleaving between parent and child processes.
- `void childProcess();`
Purpose: This where the child process will do its work. You need to print out "Child process executing" **five** times. Add a delay between each printout using the `sleep()` function. A suitable delay will cause interleaving between parent and child processes.
- `int main(int argc, char* argv[]);`
Purpose: This is where the process creation will take place. You will need to start the child process using the `fork()` function. You should then call the respective function for each process. Processes should wait for each other to complete using the `wait()` function.

```
Parent process executing
Parent process executing
Child process executing
Parent process executing
Child process executing
Parent process executing
Child process executing
Parent process executing
Child process executing
Child process executing
```

Figure 1: Example output from single execution

Important!

- You should use the specified C++ program file name.
- Make sure that your output strictly matches the example output. The line ordering should vary. There should be no extra spaces or new lines.

Upload Archive

You will need to submit the following files as an archive (zip or tar.gz) to **FitchFork**.

- process_main.cpp
- makefile